

Chapitre 3 – Android – Eléments d'interface

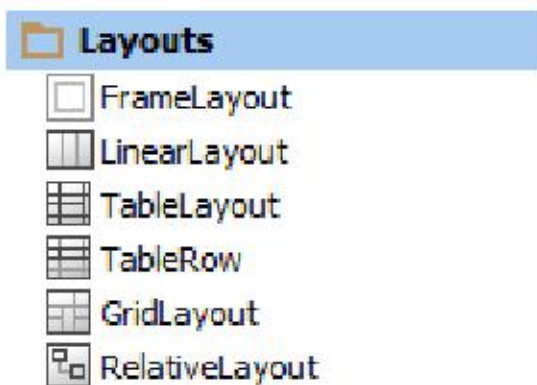
Sommaire

I. Principaux éléments de l'interface	1
II. Création statique ou dynamique d'interface	2
III. Internationalisation	4

I. Principaux éléments de l'interface

1. Layouts

Les *layouts* représentent des conteneurs plus ou moins évolués.

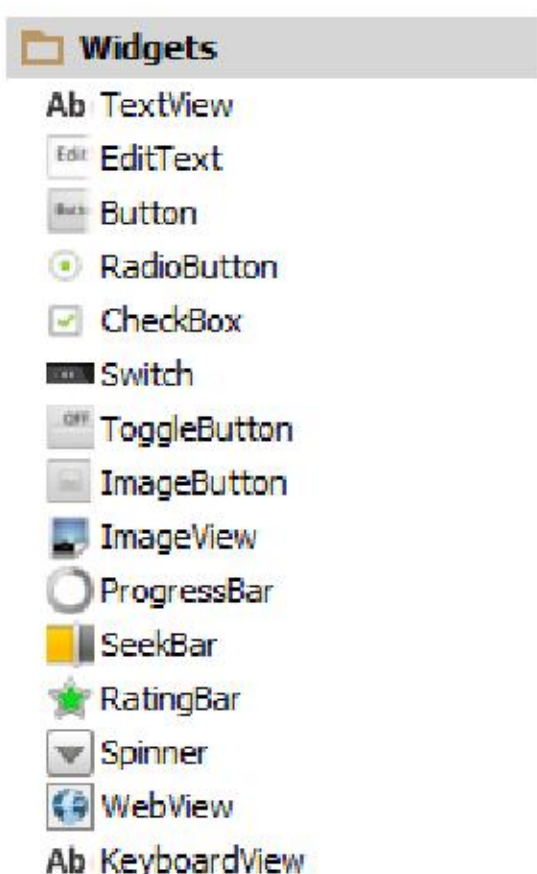


Le *LinearLayout* est celui utilisé par défaut (disposition de gauche à droite ou de haut en bas). Principales propriétés :

- orientation : *vertical* ou *horizontal*,
- layout_width : *fill_parent* = *match_parent* (remplit le parent => même largeur), ou *match_content* (ajusté à son contenu => largeur minimale),
- layout_height : *fill_parent* = *match_parent* ou *match_content*.

2. Widgets

Les *widgets* sont les éléments d'interface « élémentaires » qui ne peuvent servir de conteneur.



Principales propriétés :

- layout_width : *fill_parent* = *match_parent*, ou *fill_content*,

- `layout_height` : *fill_parent* = *match_parent* ou *fill_content*,
- `gravity` : Combinaison de *left*, *right*, *top*, *bottom* et *center* (*vertical* et/ou *horizontal*) pour positionner l'objet dans le conteneur.

II. Création statique ou dynamique d'interface

Par défaut, les interfaces sont créées statiquement, via l'éditeur, et stockées dans un fichier ressource au format *xml*. Tous les « layouts » statiques doivent être dans le dossier `<NomDuProjet>\res\layout`.

Il peut arriver qu'il faille créer dynamiquement un *layout*, ou une partie d'un *layout*. Voici les équivalents statiques / dynamiques pour créer un layout ou bien des widgets.

1. LinearLayout

Création statique (*xml*) d'un *LinearLayout* occupant tout l'écran, nommé *llLayout* :

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/llLayout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

</LinearLayout>
```

Référencement (*java*) du *LinearLayout* nommé *llLayout* :

```
LinearLayout ll=(LinearLayout) findViewById(R.id.llLayout);
```

Création dynamique (*java*) d'un *LinearLayout* occupant tout l'écran, sans nom :

```
LinearLayout ll=new LinearLayout (this);
ll.setOrientation(LinearLayout.VERTICAL);
// Remplissage du layout puis affichage
setContentView(ll);
```

2. TextView

Création statique (*xml*) d'un *TextView* intégré au *LinearLayout* précédent, nommé *tvLibelle* :

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/llLayout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <TextView
        android:id="@+id/tvLibelle"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Libellé" />

</LinearLayout>
```

Référencement (*java*) du *TextView* nommé *tvLibelle* :

```
TextView tvLibelle=(TextView) findViewById(R.id.tvLibelle);
```

Création dynamique (*java*) d'un *TextView* dans le *LinearLayout*, sans nom :


```

        TextView tvLibelle=new TextView(this);
// Définition du TextView
        tvLibelle.setText("Mon libellé");
// Intégration dans le layout
        ll.addView(tvLibelle);

```

3. EditText

Création statique (xml) d'un *EditText* intégré au *LinearLayout* précédent, nommé *etNom* :

```

...
<EditText
    android:id="@+id/etNom"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:ems="10"
    android:inputType="textPersonName"
...

```

Référencement (java) d'un *EditText* nommé *etNom* :

```

EditText etNom=(EditText) findViewById(R.id.etNom);

```

Création dynamique (java) d'un *EditText* dans le *LinearLayout*, sans nom :

```

        EditText etNom=new EditText(this);
// Définition du EditText
        etNom.setText("Mon texte");
        etNom.setSingleLine();
// Intégration dans le layout
        ll.addView(etNom);

```

SetText et *SetSingleLine* correspondent à la définition facultative des propriétés *Text* et *SingleLine*.

4. Événement click sur un Button

Les événements sont associés via l'éditeur aux objets statiques de l'interface. Pour les objets dynamiques, il faut nécessairement une association différente (dynamique elle aussi).

Étapes :

- Créer dynamiquement ou utiliser un bouton existant,
- Associer un gestionnaire de clic (Click Listener) avec *SetOnClickListener*,
- Ce gestionnaire est une classe *OnClickListener* pour lequel la méthode *onClick* doit être surchargée :

```

        Button bt=new Button(this);
// Définition du Button
        bt.setText("Mon bouton");
        bt.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                Toast.makeText(view.getContext(),"cliqué !",Toast.LENGTH_LONG).show();
            }
        });
// Intégration dans le layout
        ll.addView(bt);

```

Le gestionnaire d'événement peut être simplifié grâce au recours de « l'expression lambda » :

```

b.setOnClickListener(view -> Toast.makeText(getApplicationContext(), "Cliqué",
Toast.LENGTH_SHORT).show());

```


III.Internationalisation

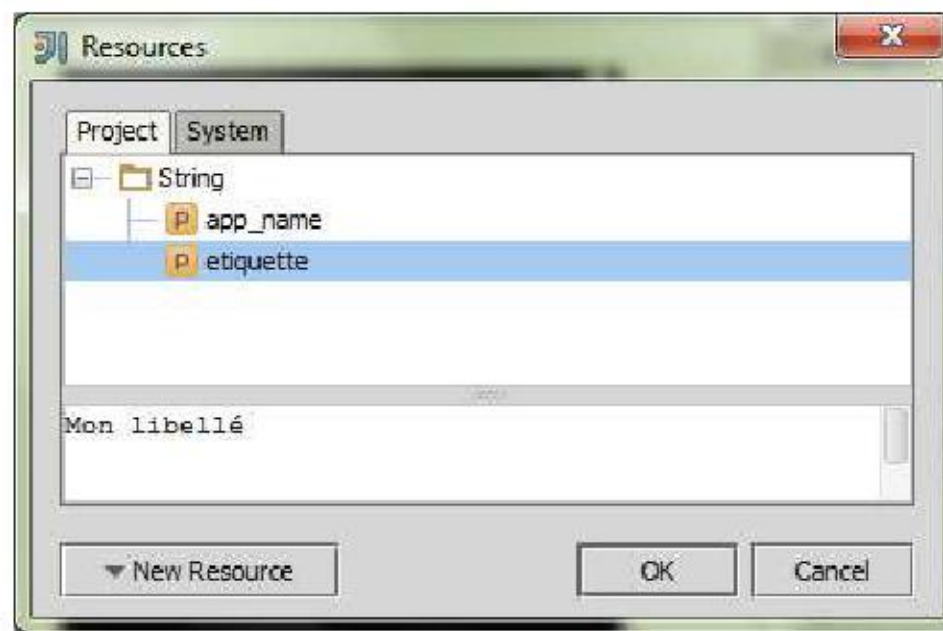
Android recommande de centraliser toutes les chaînes constantes dans un fichier de ressources, nommé *res\values\strings.xml*. *Idea* vous y incite aussi :

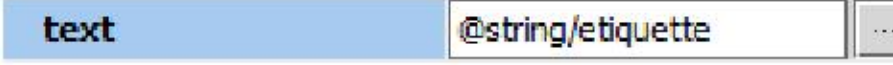
En sélectionnant un *TextView* contenant le message *Entrer votre prénom* par exemple, une petite ampoule signale que la chaîne du message est « codée en dur ». Elle invite à utiliser une ressource *@string ...* Qu'est-ce et pourquoi ?

Pour une application destinée à être utilisée dans d'autres pays, d'autres messages seront affichés. Android propose une organisation toute prête : Consulter depuis la fenêtre projet, *res, values, strings.xml* : Ce fichier sert à énumérer des valeurs associées à des noms. En créant l'arborescence *values-fr*, les chaînes décrites ici seront automatiquement utilisées à la place de celles de *values* si le mobile est en français. Etc ... Pas besoin d'aller chercher dans un autre fichier !

Pour l'affectation statique (via l'éditeur) d'une chaîne issue d'une ressource, on peut procéder ainsi :

- Sélectionner l'objet graphique : ,
- Dans la propriété *Text*, cliquer sur ... : 



- Créer / Choisir la ressource : 
- La ressource est associée : 

Le résultat se voit dans le fichier *xml* :

```
...  
<TextView  
    android:id="@+id/tvLibelle"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="@string/libelle" />  
...
```

Pour une affectation dynamique, l'affectation "en dur" :

```
tvLibelle.setText("Mon libellé");
```

Est remplacée par :

```
tvLibelle.setText(R.string.libelle);
```